MentorMatch ML Project

"People struggle to find the right mentors due to lack of structured matching systems."

In [ ]:
```
!pip install xgboost scikit-learn pandas numpy matplotlib seaborn
```

```
Requirement already satisfied: xgboost in ./venv/lib/python3.12/site
-packages (1.7.6)
Requirement already satisfied: scikit-learn in ./venv/lib/python3.1
2/site-packages (1.8.0)
Requirement already satisfied: pandas in ./venv/lib/python3.12/site-
packages (2.3.3)
Requirement already satisfied: numpy in ./venv/lib/python3.12/site-p
ackages (2.4.3)
Requirement already satisfied: matplotlib in ./venv/lib/python3.12/s
ite-packages (3.10.8)
Requirement already satisfied: seaborn in ./venv/lib/python3.12/site
-packages (0.13.2)
Requirement already satisfied: scipy in ./venv/lib/python3.12/site-p
ackages (from xgboost) (1.17.1)
Requirement already satisfied: joblib>=1.3.0 in ./venv/lib/python3.1
2/site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in ./venv/lib/py
thon3.12/site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in ./venv/lib/
python3.12/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in ./venv/lib/python3.1
2/site-packages (from pandas) (2026.1.post1)
Requirement already satisfied: tzdata>=2022.7 in ./venv/lib/python3.
12/site-packages (from pandas) (2025.3)
Requirement already satisfied: contourpy>=1.0.1 in ./venv/lib/python
3.12/site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in ./venv/lib/python3.1
2/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in ./venv/lib/pytho
n3.12/site-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in ./venv/lib/pytho
n3.12/site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in ./venv/lib/python
3.12/site-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in ./venv/lib/python3.12/si
te-packages (from matplotlib) (12.1.1)
Requirement already satisfied: pyparsing>=3 in ./venv/lib/python3.1
2/site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in ./venv/lib/python3.12/sit
e-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

In [ ]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, StackingClassi
from xgboost import XGBClassifier
from sklearn.metrics import classification_report, accuracy_score,
from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt
import seaborn as sns
```

Upload Dataset

In [ ]:
```
import pandas as pd

df = pd.read_csv('dataset.csv')
df.head()
```

Out[ ]:

	profile_id	name	email	
0	54641238e8a5986c20e095bdfbd58f9c	Brett Petersen	samantha25example.net	
1	74d242dc227f39aa211913cb302cefb9	Daniel Stewart	adonovanexample.org	Ki
2	dcaef3d5aea0edaeae6f31f65812f9da	Mary Tucker	shawn77example.com	
3	b466db7fe3f40bce14c91b9ad2991719	Sara Ramirez	ortizmichelleexample.org	
4	e08cf08bf687933f06b5ecac3e5ef572	Andre Brown	troy00example.org	Cu

Data Preprocessing

In [ ]:
```
import ast
```

```
df.fillna('', inplace=True)

# Convert string lists → proper text
df['skills'] = df['skills'].apply(lambda x: " ".join(ast.literal_ev
df['goals'] = df['goals'].apply(lambda x: " ".join(ast.literal_eval
df['needs'] = df['needs'].apply(lambda x: " ".join(ast.literal_eval

df['combined_text'] = df['skills']
df['recommend_text'] =df['skills'] + " " + df['goals']

df[['skills', 'goals', 'needs', 'combined_text']].head()
```

Out[ ]:

	skills	goals	needs	combined_text
0	User Research Microservices Adobe XD UIUX Desi...	Mentor others Lead projects	job opportunities funding partnerships	User Research Microservices Adobe XD UIUX Desi...
1	CICD Sketch AWS TensorFlow Python Big Data Dat...	Learn new skills Get promoted	technical expertise career guidance mentorship	CICD Sketch AWS TensorFlow Python Big Data Dat...
2	Spark Data Science Deep Learning Angular Micro...	Build company Advisory roles	clients job opportunities business advice	Spark Data Science Deep Learning Angular Micro...
3	React Sketch Figma R Java Prototyping Scala De...	Mentor others Specialize	funding partnerships technical expertise	React Sketch Figma R Java Prototyping Scala De...
4	Sketch PyTorch Swift Illustrator Prototyping R...	Build network Learn new skills	technical expertise network connections funding	Sketch PyTorch Swift Illustrator Prototyping R...

VISUALIZATION

In [ ]:

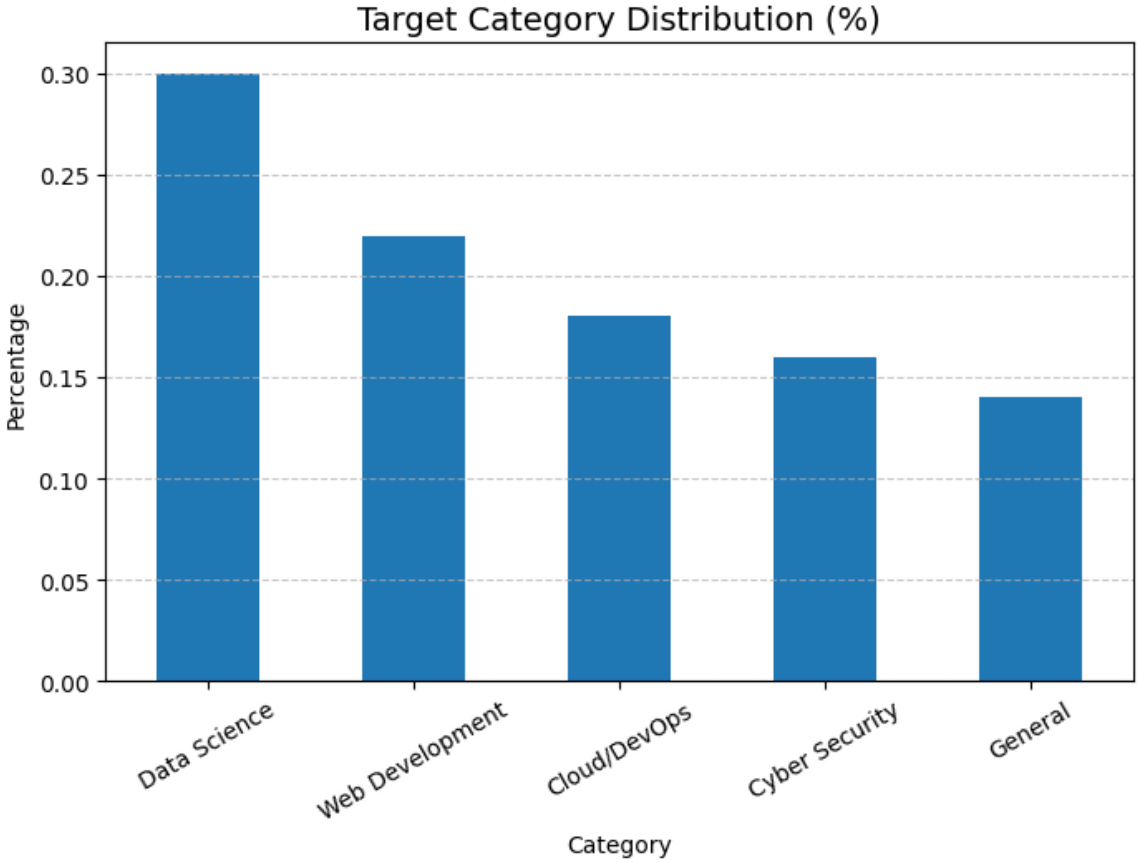
```
import matplotlib.pyplot as plt
import seaborn as sns

# Category Distribution
# plt.figure()
# df['target_category'].value_counts().plot(kind='bar')
# plt.title("Target Category Distribution")
# plt.xlabel("Category")
# plt.ylabel("Count")
# plt.xticks(rotation=45)
# plt.show()

plt.figure(figsize=(8,5))
df['target_category'].value_counts(normalize=True).plot(kind='bar')

plt.title("Target Category Distribution (%)", fontsize=14)
plt.ylabel("Percentage")
```

```
plt.xlabel("Category")
plt.xticks(rotation=30)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



TF-IDF Feature Engineering

In [ ]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import LabelEncoder

tfidf = TfidfVectorizer(max_features=2000, ngram_range=(1,1))

X = tfidf.fit_transform(df['combined_text'])

le = LabelEncoder()
y = le.fit_transform(df['target_category'])
```

Train-Test Split

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Model Training

In [ ]:
```python
def run_model(name, model):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average='weighted')

    print(f"\n{name}")
    print("Accuracy:", acc)
    print("F1 Score:", f1)

    return acc, f1, y_pred


# Logistic Regression (Baseline)
lr = LogisticRegression(max_iter=300, C=0.5)
acc_lr, f1_lr, y_pred_lr = run_model("Logistic Regression", lr)


# Random Forest
rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=12,
    random_state=42,
    class_weight='balanced'
)
acc_rf, f1_rf, y_pred_rf = run_model("Random Forest", rf)


# XGBoost
xgb = XGBClassifier(
    n_estimators=200,
    max_depth=5,
    learning_rate=0.08,
    subsample=0.8,
    colsample_bytree=0.8
)
acc_xgb, f1_xgb, y_pred_xgb = run_model("XGBoost", xgb)
```

```
Logistic Regression
Accuracy: 0.9048
F1 Score: 0.9048007572408713

Random Forest
Accuracy: 0.9218
F1 Score: 0.9221677751374918

XGBoost
Accuracy: 0.9371
F1 Score: 0.9376827228130392
```

Stacking Ensemble

In [ ]:
```python
stack_model = StackingClassifier(
```

```
    estimators=[
        ('lr', lr),
        ('rf', rf),
        ('xgb', xgb)
    ],
    final_estimator = RandomForestClassifier(max_depth=10, n_estima
    stack_method='predict_proba',
    cv=5,
    n_jobs=-1
)
acc_stack, f1_stack, y_pred_stack = run_model("Stacking", stack_mod
```

```
Stacking
Accuracy: 0.9533
F1 Score: 0.9535225308420293
```

Stacking Cross - Validation

In [ ]:
```
from sklearn.model_selection import cross_val_score

cv_scores = cross_val_score(stack_model, X, y, cv=5, scoring='f1_we

print("\nCross-Validation Results:")
print("F1 Scores for each fold:", cv_scores)
print("Mean F1 Score:", cv_scores.mean())
```

Model Evaluation

In [ ]:
```
def evaluate(name, y_true, y_pred):
    print(f"\n{name}")
    print("Accuracy:", accuracy_score(y_true, y_pred))
    print("F1 Score:", f1_score(y_true, y_pred, average='weighted')
    print(classification_report(y_true, y_pred, zero_division=0))
```

In [ ]:
```
evaluate('Logistic Regression', y_test, y_pred_lr)
evaluate('Random Forest', y_test, y_pred_rf)
evaluate('XGBoost', y_test, y_pred_xgb)
evaluate('Stacking', y_test, y_pred_stack)
```

```
Logistic Regression
Accuracy: 0.9048
F1 Score: 0.9048007572408713
              precision    recall  f1-score   support

           0       0.92      0.91      0.91      1800
           1       0.71      0.69      0.70      1600
           2       1.00      1.00      1.00      3000
           3       0.76      0.79      0.78      1400
           4       1.00      1.00      1.00      2200

    accuracy                           0.90     10000
   macro avg       0.88      0.88      0.88     10000
weighted avg       0.90      0.90      0.90     10000
```

```
Random Forest
Accuracy: 0.9218
F1 Score: 0.9221677751374918
              precision    recall  f1-score   support

           0       0.96      0.91      0.93      1800
           1       0.77      0.73      0.75      1600
           2       1.00      1.00      1.00      3000
           3       0.77      0.87      0.82      1400
           4       1.00      1.00      1.00      2200

    accuracy                           0.92     10000
   macro avg       0.90      0.90      0.90     10000
weighted avg       0.92      0.92      0.92     10000


XGBoost
Accuracy: 0.9371
F1 Score: 0.9376827228130392
              precision    recall  f1-score   support

           0       0.97      0.93      0.95      1800
           1       0.75      0.90      0.82      1600
           2       1.00      1.00      1.00      3000
           3       0.90      0.76      0.83      1400
           4       1.00      1.00      1.00      2200

    accuracy                           0.94     10000
   macro avg       0.93      0.92      0.92     10000
weighted avg       0.94      0.94      0.94     10000


Stacking
Accuracy: 0.9533
F1 Score: 0.9535225308420293
              precision    recall  f1-score   support

           0       0.97      0.95      0.96      1800
           1       0.84      0.88      0.86      1600
           2       1.00      1.00      1.00      3000
           3       0.90      0.87      0.88      1400
           4       1.00      1.00      1.00      2200

    accuracy                           0.95     10000
   macro avg       0.94      0.94      0.94     10000
weighted avg       0.95      0.95      0.95     10000
```

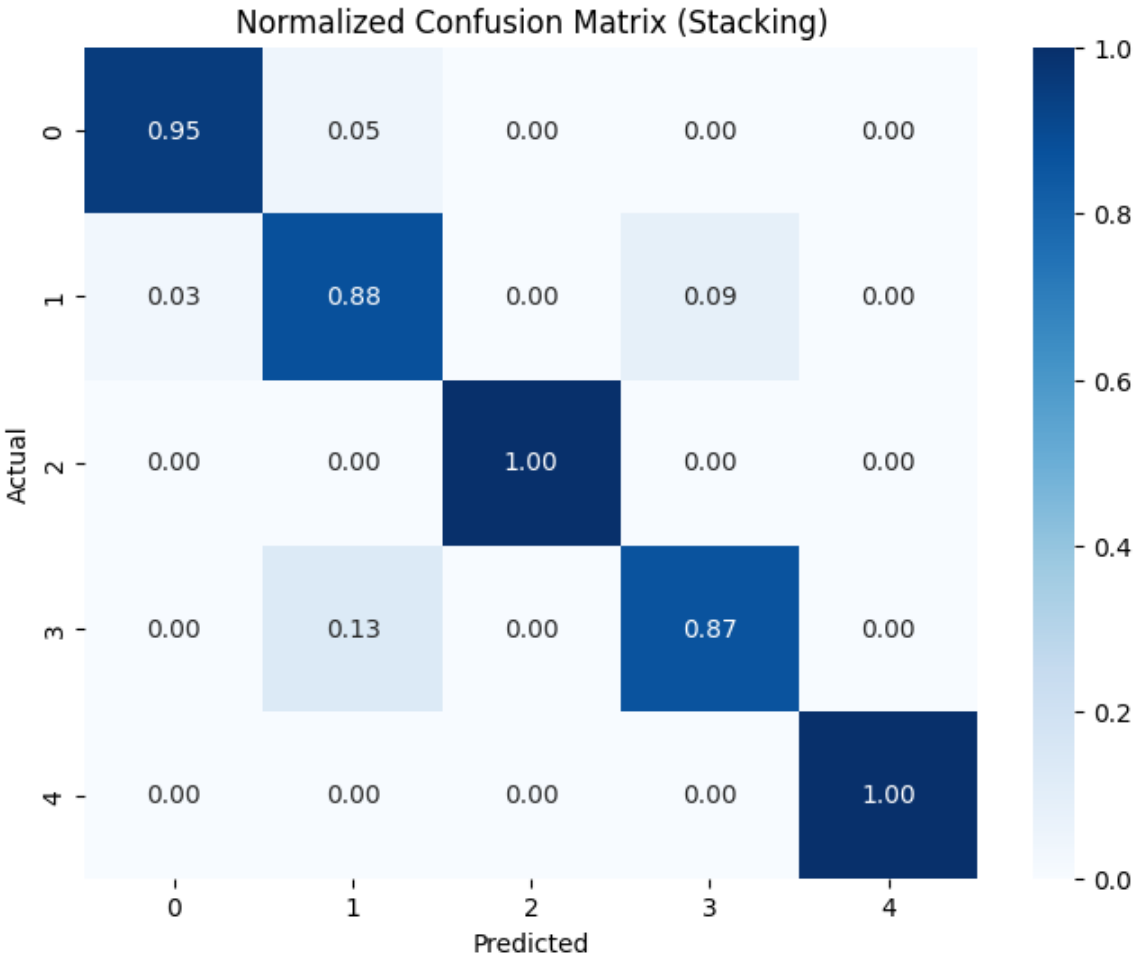
Confusion Matrix

In [ ]:
```
cm = confusion_matrix(y_test, y_pred_stack, normalize='true')

plt.figure(figsize=(8,6))
```

```
sns.heatmap(cm, annot=True, cmap='Blues', fmt=".2f")

plt.title("Normalized Confusion Matrix (Stacking)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



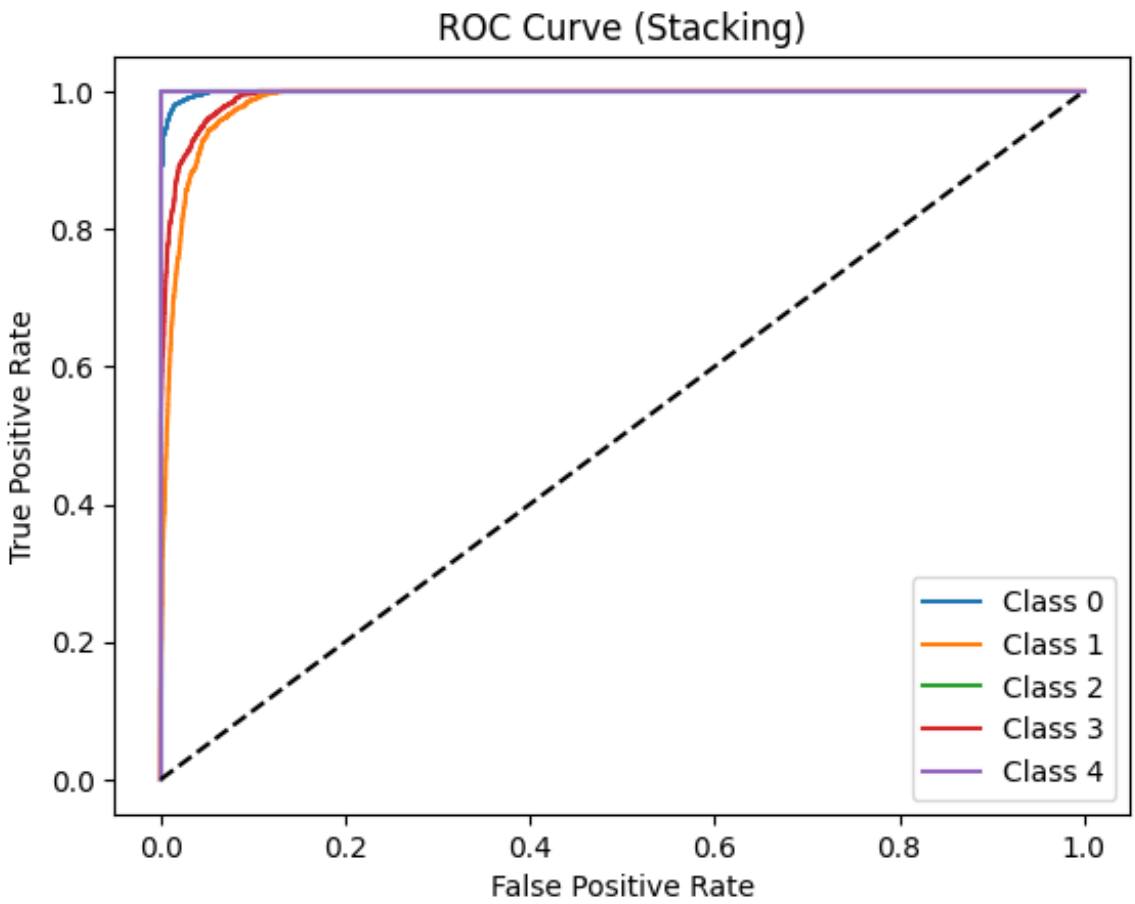
In [ ]:

```
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc

y_test_bin = label_binarize(y_test, classes=[0,1,2,3,4])
y_pred_prob = stack_model.predict_proba(X_test)

for i in range(y_test_bin.shape[1]):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_pred_prob[:, i])
    plt.plot(fpr, tpr, label=f"Class {i}")

plt.plot([0,1],[0,1],'k--')
plt.title("ROC Curve (Stacking)")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()
```



Model Comparison Graph

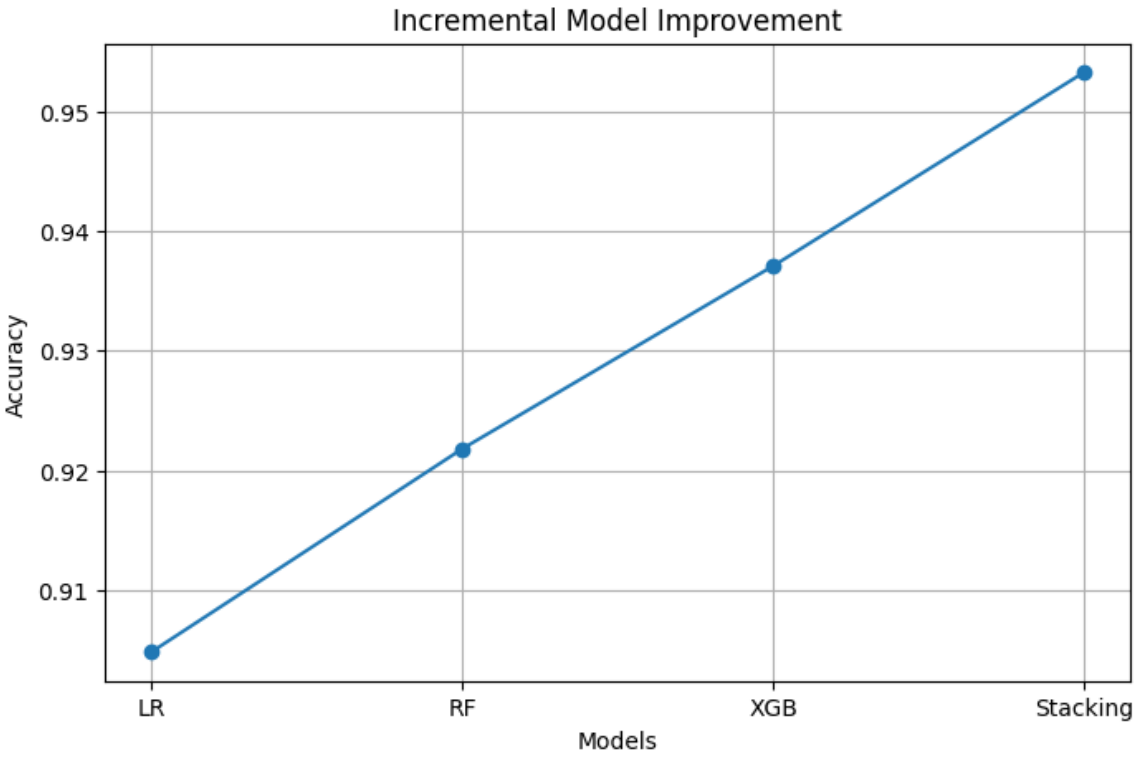
In [ ]:
```
models = ['LR', 'RF', 'XGB', 'Stacking']
accuracy = [
    accuracy_score(y_test, y_pred_lr),
    accuracy_score(y_test, y_pred_rf),
    accuracy_score(y_test, y_pred_xgb),
    accuracy_score(y_test, y_pred_stack)
]

plt.figure(figsize=(8,5))
results = {
    "LR": acc_lr,
    "RF": acc_rf,
    "XGB": acc_xgb,
    "Stacking": acc_stack
}

models = list(results.keys())
accuracy = list(results.values())

plt.figure(figsize=(8,5))
plt.plot(models, accuracy, marker='o')
plt.title("Incremental Model Improvement")
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.grid()
plt.show()
```

```
<Figure size 800x500 with 0 Axes>
```



In [ ]:
```python
X = tfidf.fit_transform(df['combined_text'])

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size

lr.fit(X_train, y_train)
rf.fit(X_train, y_train)
xgb.fit(X_train, y_train)

stack_model.fit(X_train, y_train)
```

Out[ ]:

```
▸ StackingClassifier
lr
▸ LogisticRegression
rf
▸ RandomForestClassifier
final_estimator
▸ RandomForestClassifier
```

In [ ]:
```python
from sklearn.metrics.pairwise import cosine_similarity

# ALWAYS recompute after tfidf.fit_transform
similarity_matrix = cosine_similarity(
    tfidf.transform(df['recommend_text'])
)
```

Recommendation System

In [ ]:

```
def recommend_hybrid(user_index, top_n=5):

    user_text = df.iloc[user_index]['combined_text']

    user_vector = tfidf.transform([user_text])

    pred_category_encoded = stack_model.predict(user_vector)[0]
    pred_category = le.inverse_transform([pred_category_encoded])[0

    print(f"\nPredicted Category: {pred_category}")

    #  Filter dataset by predicted category
    filtered_df = df[df['target_category'] == pred_category]
    filtered_indices = filtered_df.index.tolist()

    print(f"Total candidates considered: {len(filtered_indices)}")

    # Compute similarity only within filtered users
    scores = []
    for i in filtered_indices:
        sim = similarity_matrix[user_index][i]
        scores.append((i, sim))

    #  Sort by similarity score
    scores.sort(key=lambda x: x[1], reverse=True)

    #  Remove self and take top N
    recommendations = []
    for idx, score in scores:
        if idx != user_index:
            recommendations.append((idx, score))
        if len(recommendations) == top_n:
            break

    print("\nRecommended Mentors:\n")

    for i, score in recommendations:
        print(f"User {i} | Score: {score:.3f}")
        print("Skills:", df.iloc[i]['skills'])
        print("Goals:", df.iloc[i]['goals'])
        print("-"*40)

recommend_hybrid(1000)
```

```
Predicted Category: Data Science
Total candidates considered: 15000

Recommended Mentors:

User 5632 | Score: 0.744
Skills: Kubernetes Big Data Data Science NumPy Kotlin UIUX Design De
ep Learning Strategy CICD Sales Computer Vision User Research Adobe
XD SQL Node.js Swift Java Finance Terraform PHP Ruby Pandas
Goals: Scale impact Strategic role
----------------------------------------
User 35675 | Score: 0.742
Skills: Computer Vision User Research Rust Kubernetes Ruby Swift Pro
duct Management Prototyping Vue.js Django Express.js Big Data DevOps
Azure Pandas Node.js Data Science PyTorch FastAPI TypeScript Flask U
IUX Design
Goals: Advisory roles Build company
----------------------------------------
User 15700 | Score: 0.724
Skills: JavaScript Data Science R AWS Rust Vue.js React Product Mana
gement Computer Vision CICD Big Data Node.js Next.js Deep Learning D
ocker UIUX Design C Illustrator User Research
Goals: Strategic role Scale impact
----------------------------------------
User 3391 | Score: 0.715
Skills: Scikit-learn Microservices Azure Deep Learning Marketing Pro
totyping Big Data JavaScript CICD PyTorch Data Science SQL Scala Vu
e.js Adobe XD Node.js UIUX Design Computer Vision Kubernetes Finance
Spark R Go Pandas
Goals: Advisory roles Build company
----------------------------------------
User 48103 | Score: 0.702
Skills: Pandas Data Science Photoshop Finance Swift Computer Vision
Deep Learning AWS UIUX Design Django Marketing Analytics Node.js Pro
duct Management Strategy Big Data
Goals: Scale impact Executive position
----------------------------------------
```